

One Demonstration Is Enough for Real-World Robotic Reinforcement Learning

Yuwan Liu^{1,2,3*}, Hongze Yu^{3*}, Song Liu³, Yuhan Wang³, Junge Zhang^{1,4†},
Yaodong Yang^{5†}, Yuanpei Chen³, and Ceyao Zhang^{3,5†}

¹ National Key Laboratory of Cognition and Decision Intelligence for Complex Systems, Institution of Automation, Chinese Academy of Sciences, Beijing, China

² Beijing Academy of Artificial Intelligence, Beijing, China

³ PKU-PsiBot Joint Lab, Beijing, China

⁴ School of Artificial Intelligence, University of Chinese Academy of Sciences, Beijing, China

⁵ Institute for Artificial Intelligence, Peking University, Beijing, China

Abstract. Learning effective robot control policies on physical hardware is challenging due to costly data collection and the difficulty of reward specification. Prior work has incorporated demonstrations into reinforcement learning (RL), yet existing approaches either require large numbers of demonstrations or depend on continuous human intervention during training. To address these limitations, we present *AutoSERL*, a framework that leverages a single demonstration to fully automate the intervention process in real-world robot RL. The framework includes three complementary mechanisms to accomplish certain tasks: a *sliding window intervention* mechanism that continuously guides exploration to prevent local optima and unsafe deviations, a *safety recovery mechanism* that detects and corrects failure states via predefined trajectory recovery points, and an *intervention termination* criterion that automatically disables guidance once the policy can independently complete the task, preserving its exploration advantage. We evaluate AutoSERL on six contact-intensive manipulation tasks across two robot platforms, spanning insertion, hanging, and hinge-based tasks. AutoSERL consistently outperforms SERL initialized with 20 demonstrations, behavior cloning, and MILES — a dedicated one-shot imitation learning baseline — across all tasks while matching HIL-SERL, achieves 100% success rate on insertion tasks, and demonstrates improved robustness to positional variations, all from a single demonstration. Code and videos are available on our project website: <https://autoserl.github.io/>.

Keywords: Robotic Reinforcement Learning · One-shot Learning · Sample Efficiency

* Equal contribution.

† Corresponding authors. Email: jgzhang@nlpr.ia.ac.cn, yaodong.yang@pku.edu.cn, ceyaozhang@pku.edu.cn

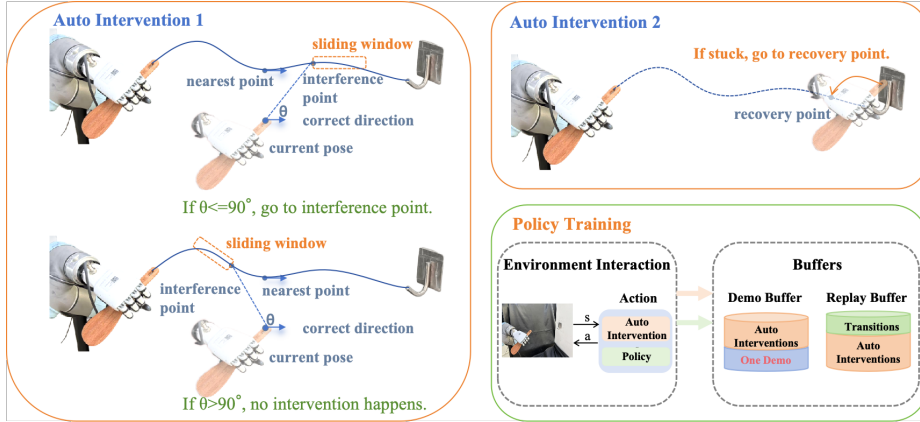


Fig. 1: Overview of AutoSERL. **Auto Intervention 1** (Sliding Window Intervention): the robot is guided to the nearest point within the sliding window only when the angle θ between the trajectory’s forward direction and the vector to that point satisfies $\theta \leq 90^\circ$, preventing the robot from being pulled back to already-visited positions. **Auto Intervention 2** (Safety Recovery Mechanism): when the robot is stuck, it is guided to the recovery point and the demonstration segment is replayed to restore progress. **Policy Training:** intervention-guided transitions and the single demonstration are stored in the Demo Buffer and Replay Buffer for policy training.

1 Introduction

Reinforcement learning (RL) has demonstrated remarkable success in simulated environments [17, 23], yet its deployment on physical robotic systems remains fundamentally constrained by two persistent challenges. First, real-world exploration carries significant inherent safety risks: unlike simulation where failures are costless, uncontrolled robot motions can cause hardware damage or environmental collisions [4]. Second, the sample inefficiency of model-free RL means that agents often fail to encounter the rare and sparse reward signals necessary for policy convergence, particularly in contact-intensive tasks where successful states occupy only a small fraction of the state space.

To address these challenges, recent work has proposed combining demonstrations with RL to accelerate real-world learning [18, 21, 26]. [14] introduces a sample-efficient framework that mixes demonstration data with online experience, significantly improving training stability on physical hardware. Building upon this, [15] incorporates human-in-the-loop intervention during training, where a human operator monitors the robot in real time and teleoperates it out of deadlocks or unsafe states. The corrected trajectories are added to the replay buffer, providing high-quality guidance that helps overcome sparse rewards and prevents unsafe exploration. This paradigm has proven effective for learning precise manipulation policies directly on real robots.

However reliance on continuous human intervention introduces a critical scalability bottleneck. Human operators are subject to cognitive fatigue, inconsis-

tent response times, and high labor costs per training hour. As tasks grow more complex and training durations extend, the requirement for constant human presence becomes logistically impractical. This raises a fundamental question: *can the benefits of human-in-the-loop training be preserved while eliminating the need for continuous human intervention?*

In this paper, we present *AutoSERL*, a framework that replaces human intervention with automated intervention mechanisms derived from a **single demonstration**. The framework consists of three complementary components to accomplish certain tasks. A *sliding window intervention* mechanism continuously monitors the agent’s end-effector pose relative to a window sliding along the demonstration trajectory, detecting deviations caused by local optima, Q-value overestimation, or environmental obstacles, and redirecting the robot accordingly. A *safety recovery mechanism* detects stagnation near interaction objects and triggers a replay-based recovery by re-executing a predefined segment of the demonstration trajectory, restoring the robot to a productive state without human intervention. An *intervention termination* criterion monitors intervention frequency during each episode and automatically disables all guidance once the policy demonstrates sufficient autonomy, allowing the agent to fully leverage RL’s exploratory advantage without unnecessary constraint.

We evaluate AutoSERL on six contact-intensive manipulation tasks spanning insertion, hanging, and hinge-based categories across two robot platforms. AutoSERL consistently outperforms [14] initialized with 20 demonstrations, behavior cloning (BC), and MILES [20] — a dedicated one-shot imitation learning method — across all tasks, achieves 100% success rate on insertion tasks, while achieving performance comparable to HIL-SERL [15], and maintains strong performance under positional variations, all from a single demonstration. Our results demonstrate that one demonstration, when carefully structured into an automated intervention system, is sufficient to match and exceed the effectiveness of both multi-demonstration RL and human-supervised training.

The main contributions of this paper are as follows: 1) we propose AutoSERL, a framework that automates real-world robot RL using only a single demonstration, eliminating the need for continuous human intervention; 2) we design three complementary mechanisms — sliding window intervention, safety recovery, and intervention termination — that together form a closed-loop automated guidance system derived entirely from one demonstration trajectory; 3) we conduct extensive real-world experiments on six contact-intensive manipulation tasks across two robot platforms, demonstrating that AutoSERL consistently outperforms multi-demonstration RL, behavior cloning, and one-shot imitation learning baselines while matching HIL-SERL.

2 Related Works

2.1 Human-in-the-Loop RL

A prominent strategy to address the challenges of real-world exploration and sparse reward signals is to integrate human expertise directly into the train-

ing loop through real-time interventions. Paradigms such as Interactive Learning [2, 13, 16, 28] and DAgger-style imitation [7, 10, 22, 29] allow human supervisors to provide corrective feedback or take over control when the agent enters a hazardous or non-productive state. By injecting this expert-induced bias, these methods transform random exploration into a structured data collection process, significantly accelerating policy convergence on physical hardware. However, these methods suffer from a "human bottleneck." The need for constant vigilance creates a scalability paradox: while humans ensure safety, their presence restricts training duration and scale due to cognitive fatigue, latency, and high costs. Instead, AutoSERL achieves the benefits of corrective supervision without the need for a human in the loop.

2.2 Safety and Exploration in Real-World RL

An established approach to ensuring safety during real-world exploration is to employ Constrained Markov Decision Processes (CMDPs) or protective safety shields that filter out hazardous actions based on predefined constraints [1, 5, 6, 8, 12, 24]. By enforcing strict operational boundaries, these methods effectively safeguard hardware against catastrophic collisions during the learning process. However, such systems often require complex multi-robot coordination and specific hardware setups. Instead, our method achieves similar levels of autonomy and safety through purely rule-based geometric heuristics, requiring no additional robotic "teacher" or supervision.

2.3 Learning from Demonstration Replay

A highly efficient approach to accelerating policy acquisition is to utilize replay-based imitation learning, which estimates the robot's pose relative to objects of interest and re-executes demonstrated action sequences [9, 19, 25, 27]. By leveraging existing expert trajectories, these methods provide a strong initial bias that bypasses the need for exhaustive random exploration in high-dimensional state spaces. Instead, our approach utilizes a replay-based recovery protocol that leverages successful action segments from the demonstration trajectory. By re-executing these validated segments upon detecting stagnation, our framework provides a computationally efficient and robust guidance mechanism.

3 Method

3.1 Preliminaries

Reinforcement learning (RL) tasks can be modeled as a Markov Decision Process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \rho, P, r, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\rho(s_0)$ is the initial state distribution, P is the state transition probability, r is the reward function, and γ is the discount factor. The learning objective is to maximize the expected cumulative reward. Our method is built upon SERL [14].

3.2 Automated Intervention Design

While SERL significantly improves sample efficiency, training can still fail on tasks with high difficulty and sparse rewards. To address this, [15] proposed HIL-SERL, which extends SERL by introducing human interventions during training to correct the robot’s actions, thereby alleviating the negative impact of sparse rewards and enabling faster and more stable learning. However, HIL-SERL requires continuous human intervention throughout training, resulting in high labor costs. To reduce this burden, we analyze the situations in which human intervention is required in HIL-SERL, and use these empirical observations as the basis for designing AutoSERL.

Taking tasks that involve interaction between a hand-held object and a single target object as an example, we identify three principal situations in which intervention is needed. The first situation occurs when the training process falls into a local optimum, where the magnitude of the output actions gradually decreases and the robot arm keeps oscillating around a certain position. The second situation occurs when the Q-value is incorrectly estimated during training, causing the policy to continuously output actions that move the robot away from the target object, which significantly reduces learning efficiency. The third situation occurs when reinforcement learning is exploring normally, but obstacles exist in the environment, potentially leading to collisions with surrounding obstacles or causing the robot to get stuck. Beyond these three situations, a residual risk exists: even when none of the above conditions are met, the robot may still become physically stuck near the target object and fail to make further progress. These four risks collectively define the design requirements for AutoSERL.

3.3 AutoSERL

AutoSERL uses a single demonstration trajectory as guidance and addresses the identified risks through three complementary mechanisms, as illustrated in Fig. 1. All tasks considered in this paper involve interaction between a hand-held object and a single target object. Before training begins, a demonstration trajectory is collected, and two recovery points are defined on it: *recover point*₀, a safe recovery target to which the robot can be guided when a safety risk occurs, and *recover point*₁, a trajectory point at which the robot is in stable contact with the interaction object. These two points can be identified by replaying the demonstration trajectory on the robot and serve as references across all three mechanisms. We also use an intervention termination threshold th_1 and an intervention start threshold th_2 .

The first and second situations—local optimum convergence and erroneous Q-value estimation—along with the third situation involving environmental obstacles, are all effectively handled by the *Sliding Window Intervention* mechanism, which actively guides the robot along the demonstration trajectory at every time step; since the trajectory is collected while maintaining a safe distance from obstacles, it naturally encodes obstacle avoidance. The residual stagnation risk is addressed by the *Safety Recovery Mechanism*, a passive fallback that detects

when the robot is stuck and restores it to a safe state. Finally, the *Intervention Termination* criterion disables all interventions once the policy becomes capable of successfully completing the task independently, preventing continuous intervention from suppressing exploration.

Sliding Window Intervention. To leverage the demonstration trajectory for guiding the training process, we define a sliding window. The length of the sliding window is equal to the index difference between *recover point*₀ and *recover point*₁ on the demonstration trajectory. The window contains a continuous segment of trajectory indices. At each time step, the system computes the distance between the current end-effector pose and those on the demonstration trajectory within the sliding window. The trajectory point with the minimum distance is selected and referred to as a potential interference point *pipoint*. When the minimum distance exceeds th_2 , further judgment is required to determine whether intervention should be performed.

To avoid pulling the robot back to past trajectory positions, a directional check is conducted. v_1 denotes the tangent vector of the trajectory at the current *cpoint* (the closest trajectory point to the end-effector), and v_2 denotes the vector from the current end-effector pose to the *pipoint*. The angle between these two vectors is computed. If the angle is greater than 90° , the *pipoint* is considered to lie on a past segment of the trajectory, and the intervention is discarded. Otherwise, motion planning is used to move the robot to the *pipoint* until the distance between the end-effector and the *pipoint* is less than th_1 .

At initialization, the end point of the sliding window is located at the first point of the demonstration trajectory. When no intervention occurs, the sliding window moves forward along the demo trajectory by one step at each time step. If the starting point of the sliding window reaches the end of the trajectory, the portion of the window that has reached the trajectory end will remain fixed, while the remaining portion that has not yet reached the end will continue to move forward. Therefore, using the sliding window for intervention can effectively prevent two situations: (1) the robot oscillating around a fixed position due to convergence to a local optimum; and (2) the policy continuously outputting actions that move the robot away from the target object due to inaccurate Q-value estimation. Moreover, since the demonstration trajectory is collected while maintaining a safe distance from environmental obstacles, this intervention strategy guides the robot toward the demonstration trajectory, thereby preventing collisions with surrounding obstacles.

Safety Recovery Mechanism. To ensure that the system can promptly recover to a controllable and safe state when safety risks occur during training, we define two recovery points on the demonstration trajectory: *recover point*₀ and *recover point*₁. *recover point*₀ is defined as a safe recovery target. When a safety risk occurs, motion planning is used to guide the robot from the current state to *recover point*₀. *recover point*₁ is defined as a trajectory point where the robot is in stable contact with the interaction object. These two recovery points can be identified by replaying the demonstration trajectory on the robot. During training, at each time step, the Euclidean distance between the current

end-effector pose and all end-effector poses along the entire demonstration trajectory is computed. The trajectory point with the minimum distance is selected and referred to as the minimum point of the full trajectory *cpoint*. The system also maintains the minimum points over the most recent l_{stag} time steps. If the distance between the current minimum point $cpoint_i$ and that from l_{stag} time steps ago $cpoint_{i-l_{stag}}$ is smaller than the threshold th_1 , and $cpoint_i$ lies between *recover point*₀ and *recover point*₁ on the demonstration trajectory, the robot is considered unable to properly interact with the object. In this case, the system first uses motion planning to move the robot to *recover point*₀, and then replays the demonstration trajectory segment between *recover point*₀ and *recover point*₁ to complete the recovery process.

Intervention Termination. Since the demonstration trajectory is not necessarily optimal, continuously applying intervention throughout training may reduce the policy’s exploration capability and limit the advantages of reinforcement learning. Therefore, in experiments without initial state randomization, if the total number of interventions in an episode is less than l_{term} and the episode successfully completes the task, all subsequent interventions are disabled.

In all tasks considered in this paper, the threshold th_1 is set to 0.005 m, the threshold th_2 is set to 0.02 m, l_{stag} is set to 20, and l_{term} is set to 10.

4 Experiments

We conduct real-world experiments to evaluate the effectiveness of AutoSERL, aiming to address the following questions: (1) Does AutoSERL achieve higher training efficiency than existing baselines? (2) How robust is AutoSERL across different seeds and under positional variations? (3) How do different heuristic hyperparameter settings affect the performance of AutoSERL? (4) What are the respective contributions of the individual components of AutoSERL to its overall performance? (5) Does AutoSERL merely imitate the demonstration trajectory, or can it learn to optimize beyond the demonstrated behavior?

4.1 Experimental Setup

Hardware and Protocol. The insertion tasks are conducted on a Franka robot arm with a parallel gripper and two wrist-mounted Intel RealSense D405 cameras. The hanging and hinge-based tasks are implemented on a UR5 robot equipped with an Inspire dexterous hand and two Intel RealSense D435 cameras. The overall setup of the two robotic platforms is shown in Fig. 2. Across all tasks, the observation consists of RGB images from the two cameras and robot proprioceptive states. For the Franka platform, proprioception includes end-effector poses, velocity, forces/torques, and gripper state. For the UR5 platform, proprioception includes end-effector poses and forces/torques. The action space is a 6D delta end-effector pose for all tasks. Training uses manually annotated binary sparse rewards. Episodes terminate upon task success or after 300 time steps.

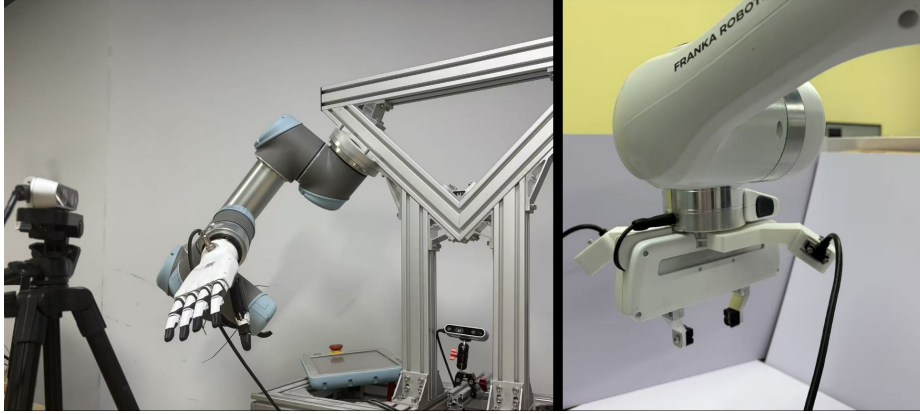


Fig. 2: Experimental setup. Left: the setup for the hanging and hinge-based tasks, consisting of a UR5 robot, an Inspire dexterous hand and two Intel RealSense D435 cameras. Right: the setup for the insertion tasks, consisting of a Franka robot and two wrist-mounted Intel RealSense D405 cameras.

During evaluation, all automatic intervention mechanisms are disabled, and each task is evaluated over 50 episodes.

Task Overview. We implement three categories of contact-intensive manipulation tasks, as illustrated in Fig. 3: insertion tasks (plug insertion and USB insertion), hanging tasks (hanging a correction tape, a hanger, and a spoon), and a hinge-based task (drawer pulling using a hook). These tasks are characterized by rich physical contact and low tolerance for execution errors, requiring high manipulation precision, accurate pose alignment, stable contact control, and strong robustness to disturbances. If the above conditions are not satisfied, the robot may easily become stuck, as shown in Fig. 4.

4.2 Results and Analysis

Training Efficiency. We compare AutoSERL with SERL [14] and HIL-SERL [15] to evaluate training efficiency. In this set of experiments, no randomization is applied to any of the task scenarios. For each task, both SERL and HIL-SERL are initialized with 20 demonstration trajectories. AutoSERL and SERL are assessed by measuring their success rates under identical training durations. As shown in Table 1, AutoSERL achieves higher success rates than SERL under the same training duration, demonstrating improved sample efficiency. Fig. 5 presents the training curves for each task. The intervention step curve shows that the number of automatic interventions gradually decreases as training progresses, indicating that the learned policy increasingly approximates the demonstrated behavior. The episode return curve shows that AutoSERL can stably accomplish the tasks without intervention after the same training duration, whereas SERL fails to achieve consistent task success. We compare AutoSERL with HIL-SERL by measuring the minimum training time required to reach a 100% success

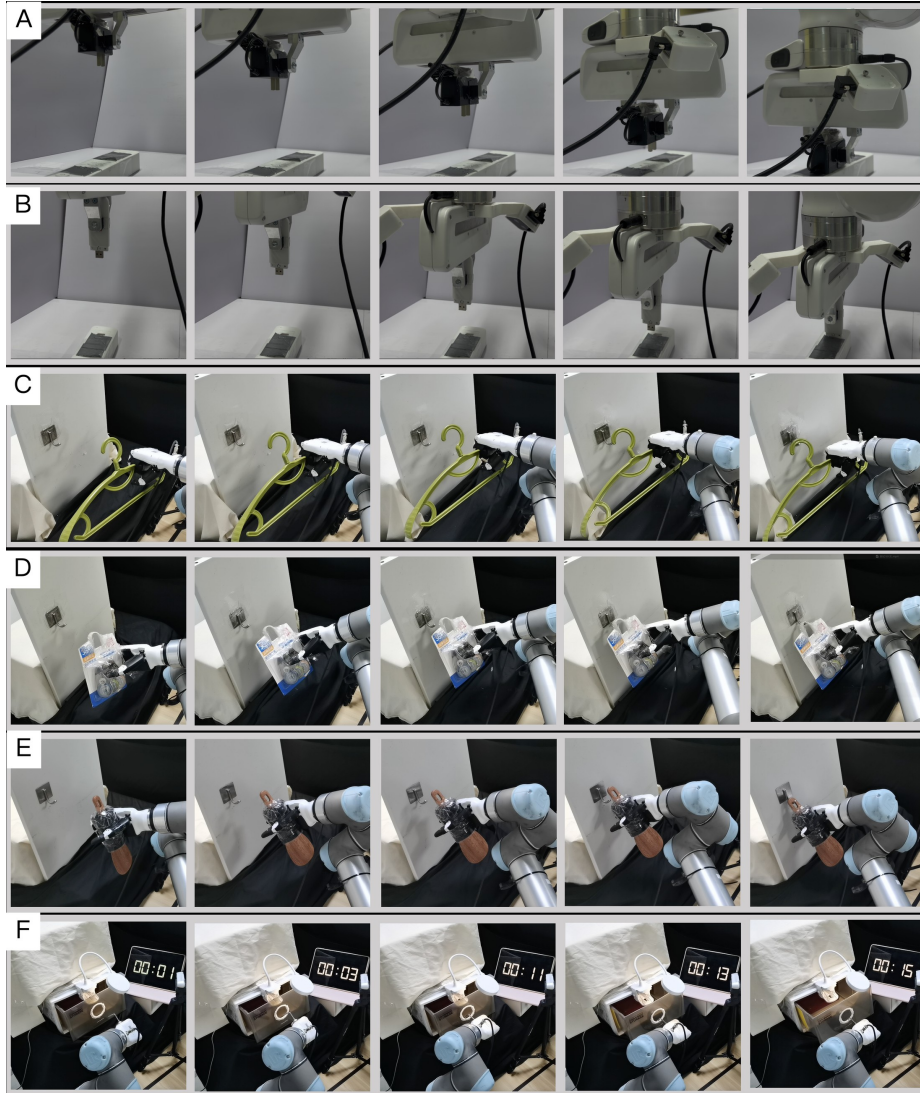


Fig. 3: Overview of the experimental tasks: (A) Plug Insertion. (B) USB Insertion. (C) Hanger Suspension. (D) Correction Tape Suspension. (E) Spoon Suspension. (F) Drawer Opening.

rate. As shown in Table 2, AutoSERL requires less or equal training time than HIL-SERL on five out of six tasks, indicating that the proposed automatic intervention mechanism provides guidance comparable to human interventions.

Comparison with Baselines. To further validate the effectiveness of AutoSERL, we compare it with BC and MILES [20] in terms of final success rate. For BC, 1 demonstration is used for the hinge-based task, 10 demonstrations for

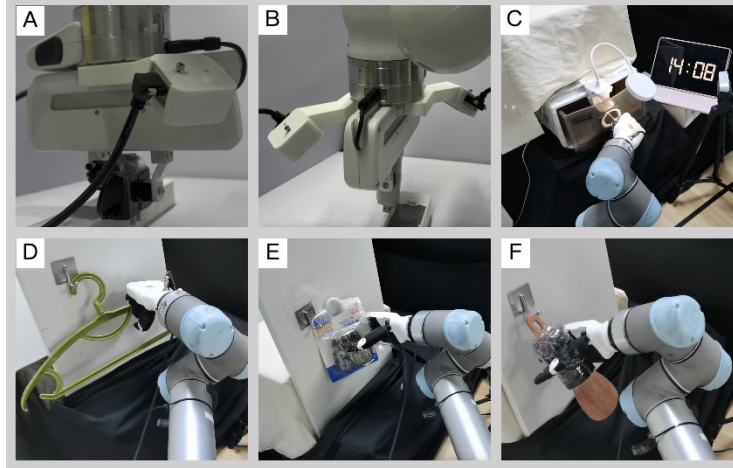


Fig. 4: Overview of the stuck cases across different tasks: (A) Plug Insertion. (B) USB Insertion. (C) Drawer Opening. (D) Hanger Suspension. (E) Correction Tape Suspension. (F) Spoon Suspension.

Table 1: Success rate of SERL and AutoSERL under the same training time budget.

Task	Training Time (min)	Success Rate	
		SERL	AutoSERL
USB Insertion	8	20/50	50/50
Plug Insertion	8	0/50	50/50
Hanger Suspension	33	0/50	50/50
Correction Tape Suspension	25	6/50	50/50
Spoon Suspension	35	0/50	50/50
Drawer Opening	45	0/50	50/50

the hanging tasks, and 20 demonstrations for the insertion tasks. For MILES, we adopt the same data augmentation randomization range as reported in the original work during data collection, namely ± 4 cm in translation and $\pm 4^\circ$ in rotation. For all methods, no task scenario randomization is introduced during evaluation. The results are summarized in Table 3. AutoSERL consistently outperforms both baselines across all tasks, demonstrating its superior performance in real-world manipulation scenarios.

4.3 Robustness Analysis

Inter-seed variance. We retrain the plug insertion task with five seeds (40–44). The success curves and summary statistics are shown in Fig. 6(a). Under all seeds, the method achieves 100% or near 100% success rates, indicating robustness to random initialization and low performance variance across different seeds.

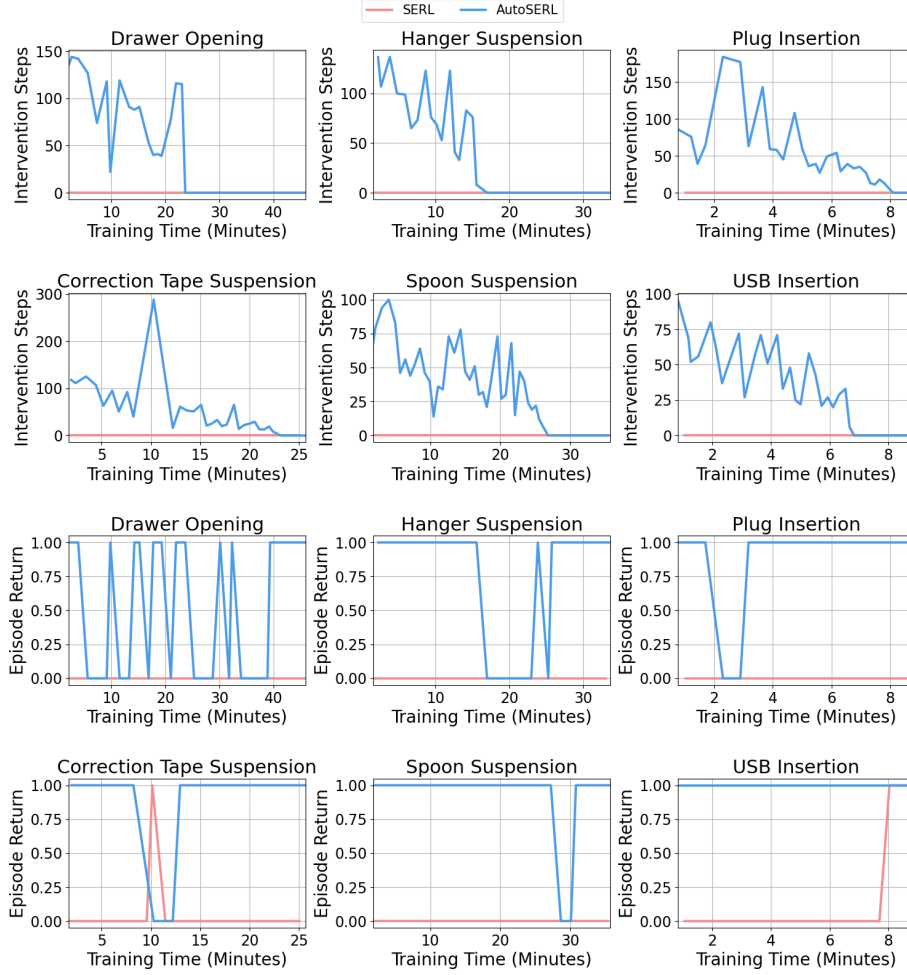


Fig. 5: Training curves of time versus intervention steps and time versus episode return for each task under SERL and AutoSERL.

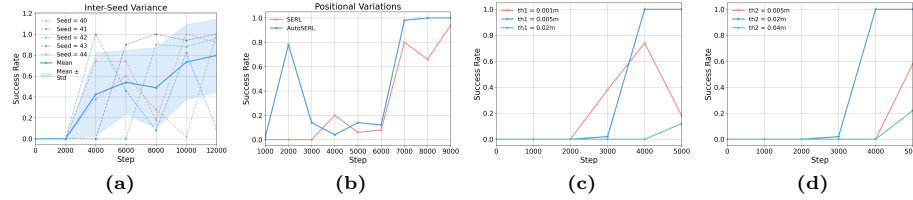
Positional variations. In the plug insertion task, we randomize the initial plug position within a ± 3 cm range in the x - y plane while keeping the socket position fixed to evaluate robustness to positional variations. For each episode, the intervention reference trajectory consists of the original demonstration trajectory concatenated with a motion-planned segment connecting the randomized initial point to the original starting point. Since positional variations alter the distribution of intervention triggers during training, all intervention mechanisms are kept active throughout the training phase of this experiment; evaluation follows the standard protocol with all interventions disabled. Under this setting, we compare SERL and AutoSERL using the same training configuration and

Table 2: Minimum training time required to achieve a 50/50 success rate for AutoSERL and HIL-SERL.

Task	Training Time (min)	
	AutoSERL	HIL-SERL
USB Insertion	8	6
Plug Insertion	8	8
Hanger Suspension	33	48
Correction Tape Suspension	25	60
Spoon Suspension	35	70
Drawer Opening	45	45

Table 3: Success rate comparison between AutoSERL and baselines.

Task	AutoSERL	BC	MILES
USB Insertion	50/50	5/50	0/50
Plug Insertion	50/50	2/50	33/50
Correction Tape Suspension	50/50	38/50	1/50
Hanger Suspension	50/50	37/50	42/50
Spoon Suspension	50/50	0/50	2/50
Drawer Opening	50/50	35/50	0/50

**Fig. 6:** Robustness and Heuristic Hyperparameter Analysis: (a) and (b) show the training curves for the plug insertion task across five random seeds and under positional variations. (c) and (d) present the training curves for the plug insertion task under different settings of hyperparameters th_1 and th_2 , respectively.

report training-time versus success-rate curves. As illustrated in Fig. 6(b), AutoSERL achieves higher success rates and more stable convergence than SERL, demonstrating improved robustness to positional variations.

4.4 Heuristic Hyperparameter Analysis

We introduce several heuristic parameters and evaluate multiple values for each on the plug insertion task, with success-rate curves over time steps shown in Fig. 6(c)(d). As shown in the figure, we study the sensitivity of th_1 and th_2 . For th_1 , we test 0.001 m and 0.02 m against the default 0.005 m; for th_2 , we

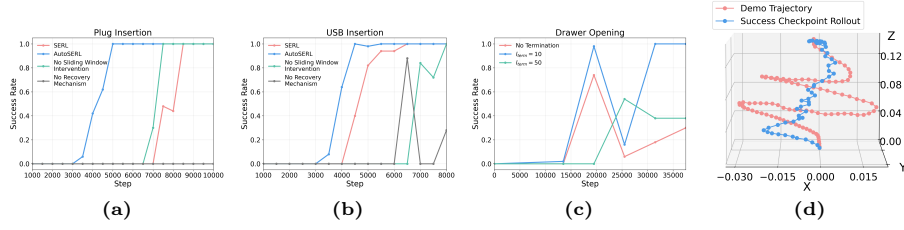


Fig. 7: Ablation study and trajectory comparison: (a) and (b) results on the plug insertion and USB insertion tasks under the *No sliding window intervention* and *No recovery mechanism* settings, respectively. (c) results on the drawer opening task under the *No intervention termination* setting. (d) results on 3D visualization of the demo trajectory and the policy rollout trajectory trained from it for the plug insertion task.

test 0.005 m and 0.04 m against 0.02 m. In both cases, the alternatives degrade performance. For th_1 , too small a value delays reaching the interference point, while too large a value weakens intervention effectiveness due to large post-intervention deviation. For th_2 , too small a value causes overly frequent interventions, reducing exploration, while too large a value leads to insufficient interventions. Overall, both parameters degrade performance when set too small or too large, and perform best within a moderate range. The stagnation window length l_{stag} defines the number of steps required for the end-effector to move beyond th_1 and is set according to the task’s action scale.

4.5 Ablation Studies

To evaluate the contribution of each component in AutoSERL, we conduct three ablation studies: (1) *No sliding window intervention*: the sliding window intervention mechanism is removed, and no interference points are used to guide the robot during training. (2) *No recovery mechanism*: the safety recovery mechanism is removed, and when the robot encounters failure states, it must rely solely on its own exploration to recover. (3) *No intervention termination*: the intervention termination criterion is removed, and the intervention continuously supervises the training process until the end.

For the *No sliding window intervention* and *No recovery mechanism* settings, we conduct experiments on the plug insertion and USB insertion tasks. The results are shown in Fig. 7(a)(b). The full AutoSERL model achieves a 100% success rate the earliest. Removing the sliding window intervention mechanism increases the number of training steps required to reach 100% success. Removing only the recovery mechanism while retaining the sliding window intervention leads to worse performance than SERL, as the sliding window strategy may guide the robot into near-failure states. Without an explicit recovery mechanism, the policy must rely solely on sparse-reward exploration to escape these states, resulting in higher failure rates and less stable learning.

For the *No intervention termination* setting, we study the mechanism on the drawer opening task under three configurations: no termination, $l_{term}=10$,

and $l_{term}=50$. The corresponding success curves are shown in Fig. 7(c). Without termination, performance peaks at approximately 19,500 steps but subsequently degrades, as continuous intervention leads to over-reliance on external corrections or frequent recovery triggers caused by misjudgments of stagnation due to small action magnitudes, thereby weakening autonomous learning. In contrast, introducing termination provides early-stage guidance with reduced exploration space, followed by a transition to autonomous reinforcement learning, improving overall performance. Moreover, $l_{term}=10$ outperforms $l_{term}=50$, indicating that overly large termination steps end intervention before sufficient guidance is obtained. Therefore, the mechanism is necessary, and its hyperparameters need to balance insufficient guidance and over-constrained learning.

4.6 Policy and Demonstration Trajectory Comparison

Taking the plug insertion task as an example, we collect a non-optimal demonstration trajectory of length 99 and train the policy using this trajectory. Rolling out the first checkpoint that achieves a 50/50 success rate yields a trajectory of length 54. The 3D visualization of both trajectories is shown in the Fig. 7(d). This suggests that the policy goes beyond simple imitation and performs trajectory-level optimization over the demonstration.

5 Discussions and Limitations

Although AutoSERL shows strong performance in real-world manipulation tasks, several limitations remain. First, for tasks with highly diverse failure modes, AutoSERL’s recovery mechanism cannot recover from failures effectively during training because it relies on information from only a single trajectory. FARL [11] and UniIntervene [3] train their recovery policies using multiple trajectories. Moreover, UniIntervene points out that when the encountered failure situation falls outside the training distribution, the reliability of recovery can be significantly reduced. Therefore, a promising research direction is to leverage more data to train a more robust recovery policy, enabling automated real-world robot RL for a wider range of tasks. Second, the current framework is limited to tasks with a 6D delta end-effector pose action space, and generalizing automated real-world robotic RL to higher-dimensional action spaces remains future work.

6 Conclusion

We propose AutoSERL, a real-world RL method that enables training through automatic intervention using only a single demonstration trajectory. AutoSERL incorporates sliding window intervention, safety recovery, and intervention termination to ensure safe and stable real-world reinforcement learning. AutoSERL outperforms multi-demonstration RL, behavior cloning, and one-shot imitation learning baselines across six contact-intensive tasks spanning three task categories while matching HIL-SERL. We hope this work will inspire future research

on automated real-world robotic RL and enable its extension to tasks with more diverse failure modes and higher-dimensional action spaces.

Acknowledgements

We gratefully acknowledge Borong Zhang for his valuable suggestions. We also thank Yu Li and Yishuai Cai for support in the initial setup of the Franka robotic arm system. This work is supported by the Chinese Academy of Sciences Project for Young Scientists in Basic Research (Grant No. YSBR-107) and the National Natural Science Foundation of China (Grant Nos. 62376013 and 62561160152).

References

1. Alshiekh, M., Bloem, R., Ehlers, R., Könighofer, B., Niekum, S., Topcu, U.: Safe reinforcement learning via shielding (2017), <https://arxiv.org/abs/1708.08611>
2. Chen, Y., Tian, S., Liu, S., Zhou, Y., Li, H., Zhao, D.: Conrft: A reinforced fine-tuning method for vla models via consistency policy (2025), <https://arxiv.org/abs/2502.05450>
3. Deng, H., Gao, Y., Lin, Y., Liu, H., Wu, Z., Wang, Z.: Uniintervene: Agentic intervention for efficient real-world reinforcement learning. arXiv preprint arXiv:2606.12372 (2026)
4. Dulac-Arnold, G., Mankowitz, D., Hester, T.: Challenges of real-world reinforcement learning. arXiv preprint arXiv:1904.12901 (2019)
5. Fisac, J.F., Akametalu, A.K., Zeilinger, M.N., Kaynama, S., Gillula, J., Tomlin, C.J.: A general safety framework for learning-based control in uncertain robotic systems (2018), <https://arxiv.org/abs/1705.01292>
6. Gillula, J.H., Tomlin, C.J.: Guaranteed safe online learning via reachability: tracking a ground target using a quadrotor. In: 2012 IEEE International Conference on Robotics and Automation. pp. 2723–2730 (2012). <https://doi.org/10.1109/ICRA.2012.6225136>
7. Hoque, R., Balakrishna, A., Novoseller, E., Wilcox, A., Brown, D.S., Goldberg, K.: Thriftydagger: Budget-aware novelty and risk gating for interactive imitation learning (2021), <https://arxiv.org/abs/2109.08273>
8. Hu, K., Shi, H., He, Y., Wang, W., Liu, C.K., Song, S.: Robot trains robot: Automatic real-world policy adaptation and learning for humanoids (2025), <https://arxiv.org/abs/2508.12252>
9. Johns, E.: Coarse-to-fine imitation learning: Robot manipulation from a single demonstration (2021), <https://arxiv.org/abs/2105.06411>
10. Kelly, M., Sidrane, C., Driggs-Campbell, K., Kochenderfer, M.J.: Hg-dagger: Interactive imitation learning with human experts (2019), <https://arxiv.org/abs/1810.02890>
11. Li, H., Lei, K., Zang, S., Hu, K., Liang, Y., An, B., Li, X., Xu, H.: Failure-aware rl: Reliable offline-to-online reinforcement learning with self-recovery for real-world manipulation. arXiv preprint arXiv:2601.07821 (2026)
12. Li, S., Bastani, O.: Robust model predictive shielding for safe reinforcement learning with stochastic dynamics (2020), <https://arxiv.org/abs/1910.10885>

13. Liu, H., Nasiriany, S., Zhang, L., Bao, Z., Zhu, Y.: Robot learning on the job: Human-in-the-loop autonomy and learning during deployment (2023), <https://arxiv.org/abs/2211.08416>
14. Luo, J., Hu, Z., Xu, C., Tan, Y.L., Berg, J., Sharma, A., Schaal, S., Finn, C., Gupta, A., Levine, S.: Serl: A software suite for sample-efficient robotic reinforcement learning. In: 2024 IEEE International Conference on Robotics and Automation (ICRA). pp. 16961–16969. IEEE (2024)
15. Luo, J., Xu, C., Wu, J., Levine, S.: Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics* **10**(105), eads5033 (2025)
16. Mandlekar, A., Xu, D., Martín-Martín, R., Zhu, Y., Fei-Fei, L., Savarese, S.: Human-in-the-loop imitation learning using remote teleoperation (2020), <https://arxiv.org/abs/2012.06733>
17. Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., Riedmiller, M.: Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602* (2013)
18. Nair, A., McGrew, B., Andrychowicz, M., Zaremba, W., Abbeel, P.: Overcoming exploration in reinforcement learning with demonstrations. In: 2018 IEEE international conference on robotics and automation (ICRA). pp. 6292–6299. IEEE (2018)
19. Palo, N.D., Johns, E.: On the effectiveness of retrieval, alignment, and replay in manipulation (2023), <https://arxiv.org/abs/2312.12345>
20. Papagiannis, G., Johns, E.: Miles: Making imitation learning easy with self-supervision. *arXiv preprint arXiv:2410.19693* (2024)
21. Rajeswaran, A., Kumar, V., Gupta, A., Vezzani, G., Schulman, J., Todorov, E., Levine, S.: Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. *arXiv preprint arXiv:1709.10087* (2017)
22. Ross, S., Gordon, G.J., Bagnell, J.A.: A reduction of imitation learning and structured prediction to no-regret online learning (2011), <https://arxiv.org/abs/1011.0686>
23. Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., et al.: Mastering the game of go with deep neural networks and tree search. *nature* **529**(7587), 484–489 (2016)
24. Thananjeyan, B., Balakrishna, A., Nair, S., Luo, M., Srinivasan, K., Hwang, M., Gonzalez, J.E., Ibarz, J., Finn, C., Goldberg, K.: Recovery rl: Safe reinforcement learning with learned recovery zones (2021), <https://arxiv.org/abs/2010.15920>
25. Valassakis, E., Papagiannis, G., Palo, N.D., Johns, E.: Demonstrate once, imitate immediately (dome): Learning visual servoing for one-shot imitation learning (2022), <https://arxiv.org/abs/2204.02863>
26. Vecerik, M., Hester, T., Scholz, J., Wang, F., Pietquin, O., Piot, B., Heess, N., Rothörl, T., Lampe, T., Riedmiller, M.: Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards. *arXiv preprint arXiv:1707.08817* (2017)
27. Wen, B., Lian, W., Bekris, K., Schaal, S.: You only demonstrate once: Category-level manipulation from single visual demonstration (2022), <https://arxiv.org/abs/2201.12716>
28. Wu, P., Shentu, Y., Liao, Q., Jin, D., Guo, M., Sreenath, K., Lin, X., Abbeel, P.: Robocopilot: Human-in-the-loop interactive imitation learning for robot manipulation (2025), <https://arxiv.org/abs/2503.07771>

29. Xu, X., Hou, Y., Xin, C., Liu, Z., Song, S.: Compliant residual dagger: Improving real-world contact-rich manipulation with human corrections (2025), <https://arxiv.org/abs/2506.16685>

Appendix

A Learning Details

Our training framework is based on SERL [14]. Following SERL, we maintain both a demo buffer and a replay buffer for data storage. The demo buffer contains one pre-collected demonstration trajectory as well as transitions from automatic interventions during training. The replay buffer stores transitions collected during training together with transitions from automatic interventions. The hyperparameters used during training are summarized in Table A.1.

B Task Details

We implement six tasks across three categories. The maximum episode length for all tasks is set to 300 time steps. For tasks implemented with the UR5 robot, the action scale is 0.005 m for translation and 0.05 rad for rotation. For tasks implemented with the Franka robot, the action scale is 0.01 m for translation and 0.06 rad for rotation. The translation ranges of all tasks relative to the initial pose are listed in Table A.2, and the rotation ranges are listed in Table A.3.

The insertion tasks demand high-precision control in constrained contact settings. In both USB insertion and plug insertion, the initial distance between the gripper and the target configuration is approximately 15 cm. Success is defined as complete insertion of the object into the corresponding port or socket.

The hanging tasks require fine-grained control of both pose and trajectory to ensure correct geometric constraint interactions between the object’s hole and the hook. The initial end-effector distance from the target hanging configuration is approximately 15 cm for the correction tape and spoon tasks, and approximately 20 cm for the hanger task. The task is considered successful when the hole of the correction tape or spoon, or the curved part of the hanger, establishes contact with the curved region of the hook.

The hinge-based task consists of drawer pulling. The robot must first hook the drawer handle and then pull the drawer outward by 5 cm. The initial distance between the hook and the handle is approximately 10 cm. Success is achieved when the drawer is displaced by 5 cm. The main challenge lies in maintaining stable contact between the hook and the handle after initial engagement, while the drawer moves under the kinematic constraint of the hinge joint.

Table A.1: Training hyperparameters.

Parameter	Value
Proprio Encoder Size	64
Policy MLP Size	256×256
Critic MLP Size	256×256
Discount Factor	0.97
Optimizer	Adam
Learning Rate	3×10^{-4}
Batch Size	256

Table A.2: Translation ranges relative to the initial pose for all tasks.

Task	X (m)	Y (m)	Z (m)
USB Insertion	[-0.03, 0.03]	[-0.03, 0.03]	[-0.16, 0.01]
Plug Insertion	[-0.03, 0.03]	[-0.03, 0.03]	[-0.16, 0.01]
Correction Tape Suspension	[-0.04, 0.04]	[-0.04, 0.04]	[-0.16, 0.01]
Hanger Suspension	[-0.04, 0.04]	[-0.04, 0.04]	[-0.21, 0.01]
Spoon Suspension	[-0.04, 0.04]	[-0.04, 0.04]	[-0.15, 0.01]
Drawer Opening	[-0.01, 0.01]	[-0.035, 0.01]	[-0.1, 0.04]

Table A.3: Rotation ranges relative to the initial pose for all tasks.

Task	X (rad)	Y (rad)	Z (rad)
USB Insertion	[-0.01, 0.01]	[-0.01, 0.01]	[-0.01, 0.01]
Plug Insertion	[-0.01, 0.01]	[-0.01, 0.01]	[-0.01, 0.01]
Correction Tape Suspension	[-0.0225, 0.0225]	[-0.0225, 0.0225]	[-0.0225, 0.0225]
Hanger Suspension	[-0.0225, 0.0225]	[-0.0225, 0.0225]	[-0.0225, 0.0225]
Spoon Suspension	[-0.0225, 0.0225]	[-0.0225, 0.0225]	[-0.0225, 0.0225]
Drawer Opening	[-0.0225, 0.0225]	[-0.0225, 0.0225]	[-0.0225, 0.0225]